

Computer Graphics (CSIT 401)

IV semester, BScCSIT

Compiled by **Ankit Bhattarai**
Ambition Guru College

Syllabus

Unit	Contents	Hours	Remarks
1.	Introduction to Computer Graphics	5	
2.	Scan Conversion Algorithms	7	<i>Very Important</i>
3.	2D Transformations and Viewing	6	<i>Very Important</i>
4.	3D Transformations and Viewing	5	
5.	3D Object Representation	6	
6.	Solid Modeling	3	
7.	Visible Surface Detection	5	
8.	Illumination, Shading and Rendering Techniques	5	
9.	OpenGL and Graphics Programming	3	

Unit 6

Solid Modeling

(3 hrs.)

Solid Modeling Concepts, Sweep representations, Boundary and spatial-partitioning representation, Binary Space Partition Trees (BSP), Octree, Ray Tracing, Rasterization vs ray tracing

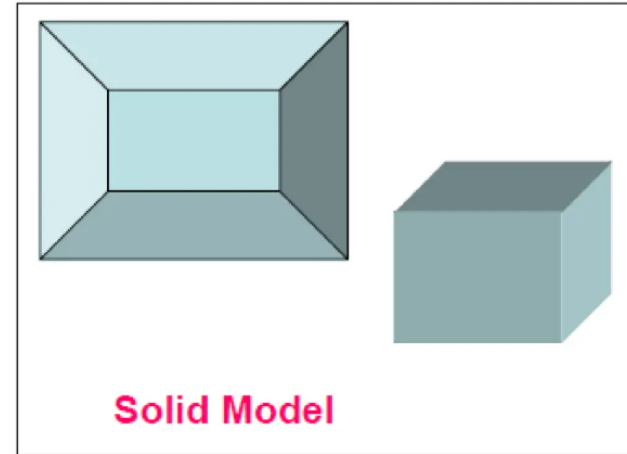
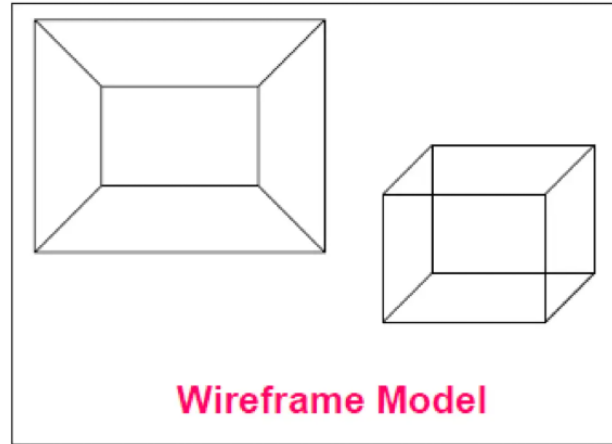
Unit 6

Section 1: Solid Modeling, Advantages, Common Representations

Solid Model

- In computer graphics, solid modeling represents the most advanced of the three primary 3D modeling types: while **wireframe** models define edges with lines and **surface** models define an empty outer shell, **solid** models define the entire volume of an object, treating it as a continuous mass rather than a hollow representation.
- Solid model of an object is a more complete representation than its surface (wireframe) model. It provides more topological information in addition to the geometrical information which helps to represent the solid unambiguously.

Solid Model

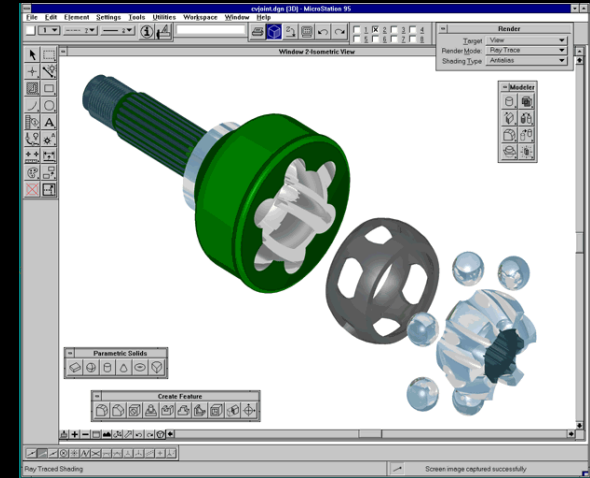
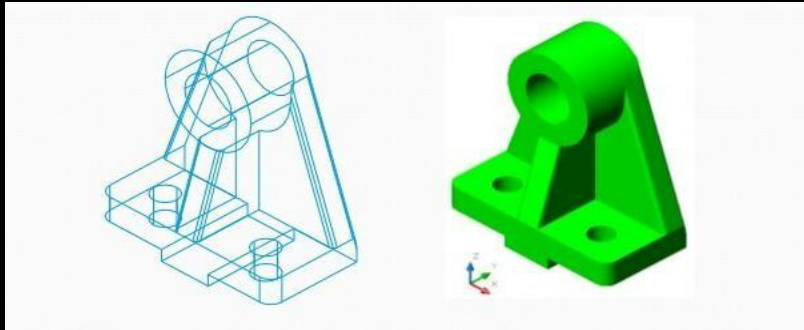


Solid Modeling

- Solid modeling is essential for visualizing complex objects, simulating their behavior, and manufacturing them through techniques like 3D printing or computer-aided manufacturing (CAM).
- Two types of solid modeling methods used in modern CAD systems are boundary representation (B-Rep) and construction solids geometry (CSG).
- Solid modeling are used extensively in various fields such as engineering, architecture, animation, and video games.

Solid Modeling

- In the solid Modeling, the solid definition include vertices(nodes), edges, surfaces, weight, and volume. The model is a complete and unambiguous representation of a precisely enclosed and filled volume.



Solid Modeling

Solid modeling is a consistent set of principles for mathematical and computer modeling of three-dimensional solids.

- A mathematical technique for representing solid objects.
- An object with following specification is called solid: Vertices, edges, surfaces, weight, and volume.

Solid modeling is based on complete, valid and unambiguous geometric representation of physical object.

- **Complete:** points in space can be classified. (inside/outside).
- **Valid:** Vertices, edges, faces are connected properly.
- **Unambiguous:** there can be only one interpretation of object.

Advantages of Solid Modeling

- **Complete Representation:** Combines geometry with topology to create an unambiguous, "watertight" model that wireframes and surfaces cannot match.
- **Physical Intelligence:** Enables instant calculation of mass properties like volume, weight, and center of gravity.
- **Manufacturing Ready:** Provides the precise data needed for CIM (Computer-Integrated Manufacturing) and automated toolpath generation.
- **Functional Accuracy:** Allows for interference checking and automated assembly testing, leading to better manufacturing automation.

Why Solid Modeling ?

- Accurate representation of 3D objects with internal structures.
- Facilitates design exploration and rapid iteration.
- Enhances visualization and communication of design concepts.
- Enables analysis and simulation of physical phenomena.
- Essential for computer-aided manufacturing (CAM) processes.
- Parametric modeling for intelligent and reusable designs.
- Supports assembly and interference checking for complex systems.
- Provides valuable documentation and archiving of design intent.

Common representations in solid modeling are:

1. **Spatial Enumeration:** In this simplest form of 3D volumetric raster model, a section of 3D space is described by a matrix of evenly spaced cubic volume elements called voxels.
2. **Cell Decomposition:** This is a hierarchical adaptation of spatial enumeration. 3D space is sub-divided into cells. Cells could be of different sizes. These simple cells are glued together to describe a solid object.
3. **Boundary Representation:** The solid is represented by its boundary which consists of a set of faces, a set of edges and a set of vertices as well as their topological relations.
4. **Sweep Methods:** In this technique a planar shape is moved along a curve. Translational sweep can be used to create prismatic objects and rotational sweep could be used for axisymmetric components.
5. **Primitive Instancing:** This modeling scheme provides a set of possible object shapes which are described by a set of parameters. Instances of object shape can be created by varying these parameters.
6. **Constructive Solid Geometry (CSG):** Primitive instances are combined using Boolean set operations to create complex object.

Unit 6

Section 2: Basis of Solid Modeling, Boundary representation, Sweep representation

Basics of Solid Modeling

Basics of solid modeling theory

- Geometry , topology
 - (x,y,z) coordinates of vertices : geometry
 - Connectivity matrix: topology
- Geometric closure

- **Set theory and Operations**

A set is a collection of objects , in the context of geometric representation the basic element of a set is a point . Two sets are commonly represented.

- W universal set containing all the elements of all sets
- $\emptyset$ null set containing no elements.
 - ✓ $A \subset B$, A is a subset of B
 - ✓ Inequality equality
 - ✓ Regularized Boolean set operations:
 - ✓ Union difference intersection etc

Solid representation

- Divide Euclidean space into two regions interior and exterior to it separated from each other by the boundary of the object.
- **Idea:** physical object is enclosed by a set of faces, which themselves belong to closed and orientable surfaces.
- Information on both geometric and topological elements is stored in the **B- rep** database.
- Euler's law gives a quantitative relationship among faces edges vertices faces etc.

NOTE: *Euclidean space refers to the classical mathematical space introduced by the ancient Greek mathematician Euclid, characterized by flat geometry and following the principles of Euclidean geometry.*

Properties of Solid models :

- Bounded boundary limits and contain the interior of the solid
- Homogenously 3D no dangling edges or faces presented boundary is always in contact with the interior of the solid
- Finite solid is not infinite in size can be described by a limited amount of information

Boundary Representation

Euler's Formula

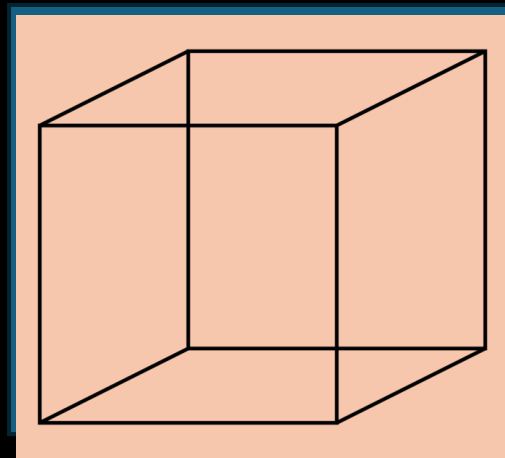
For any polyhedron *that doesn't intersect itself*, the

- Number of **F**aces
- plus the Number of **V**ertices (corner points)
- minus the Number of **E**edges always equals 2

This can be written: **$F + V - E = 2$**

Boundary Representation

Try it on the cube:



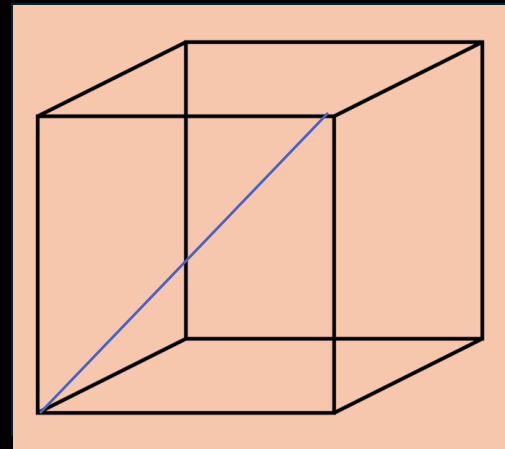
A cube has **6 Faces**, **8 Vertices**, and **12 Edges**,

so:

$$6 + 8 - 12 = 2$$

Boundary Representation

Try it on the cube:



A cube has **7 F**aces, **8 V**ertices, and **13 E**dges,

so:

$$7 + 8 - 13 = 2$$

Boundary Representation

Name		Faces	Vertices	Edges	$F+V-E$
Tetrahedron		4	4	6	2
Cube		6	8	12	2
Octahedron		8	6	12	2
Dodecahedron		12	20	30	2
Icosahedron		20	12	30	2

Boundary representation

Boundary representation

- Boundary Representation, or B-Rep, is a method of solid modeling that defines an object by its limits specifically where the "solid" ends and the "empty space" begins.
- It describes *a solid as a collection of connected surfaces that perfectly enclose a volume*. If the surfaces aren't "watertight," it isn't a valid B-Rep solid.

Data Structure: It stores two types of information:

Geometry: The mathematical description of the shapes (e.g., the exact curve of a surface or the coordinates of a point).

Topology: The "map" of how these shapes connect (which edges meet at which vertex, and which faces share which edge).

- **The Hierarchy:** A B-Rep model is organized from the ground up:

Vertices (Points) → Edges (Lines/Arcs) → Faces (Surfaces) → Shells (The closed skin).

Sweep Representation

Sweep Representation

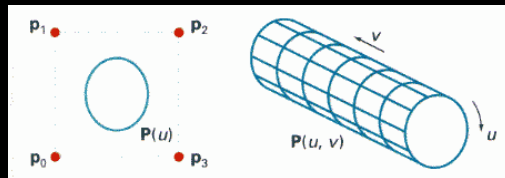
- Sweep representations create solid by moving a 2D shape (triangle, rectangle, polygon etc.) according to predefined rule (translating, rotating).
- Sweep representations are used to construct 3D object from 2D shape that have some kind of symmetry.
- For example, a prism can be generated using a translational sweep and rotational sweeps can be used to create curved surfaces like an ellipsoid or a torus.

There are two types sweep representation:

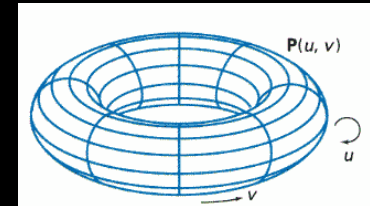
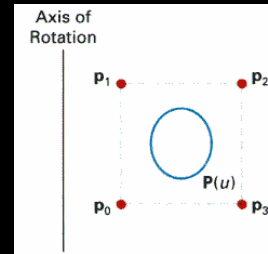
1. Translational Sweep
2. Rotational Sweep

Sweep Representation

- The former are known as extruded solids and are created via linear or translational **sweep**; the latter are solids of revolution which can be created via rotational **sweep**. **Sweeping** is based on the notion of moving a point, curve, or a surface along a given path.
- Sweep representations *mean sweeping a 2D surface in 3D space to create an object. However, the objects created by this method are usually converted into polygon meshes and/or parametric surfaces before storing.*



A Translational Sweep



A Rotational Sweep:

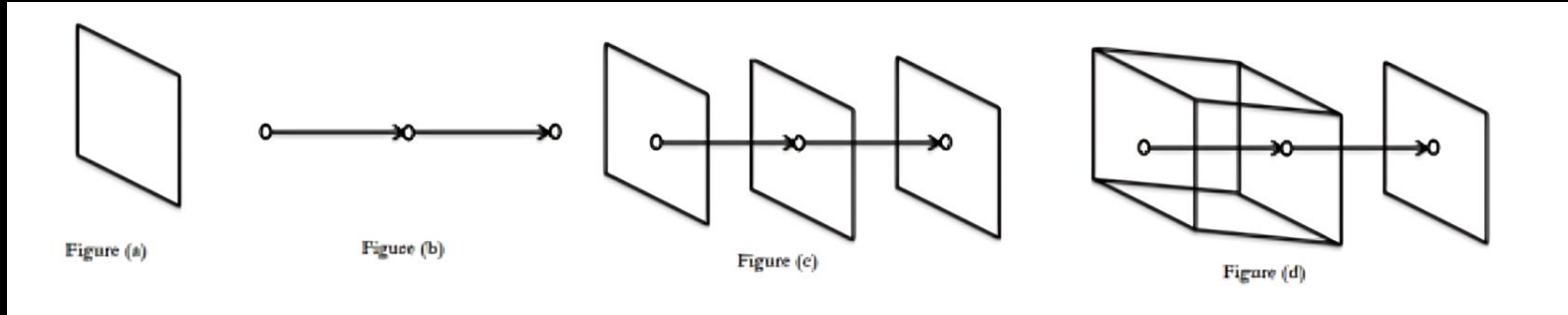
Sweep Representation: Translational Sweep

- A 2D shape is translated by a predefined translational vector. Example: following figure shows a translational sweep of rectangle.

Steps:

1. Define a shape as a polygon vertex table as shown in figure (a).
2. Define a sweep path as a sequence of translation vectors figure (b).
3. Translate the shape; continue building a vertex table figure (c).
4. Define a surface table figure (d).

Sweep Representation: Translational Sweep

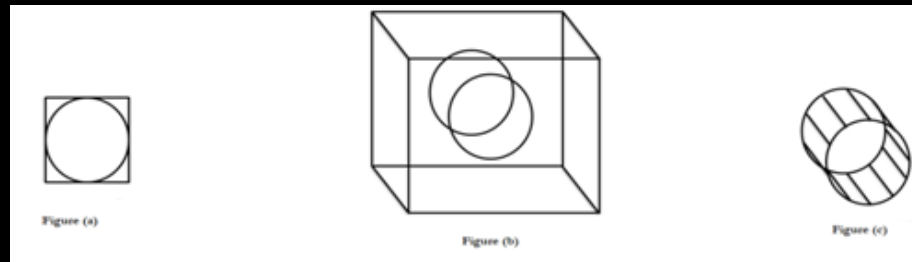


Sweep Representation: Rotational Sweep

- A 2D shape is rotated around a predefined rotational axis. Example: following figure shows a rotational sweep.

Steps:

1. Define a shape as a polygon vertex table as shown in figure (a).
2. Define a sweep path as a sequence of rotations.
3. Rotate the shape; continue building a vertex table as shown in figure (b).
4. Define a surface table as shown in figure (c).



Unit 6

Section 3: Constructive Solid Geometry (CSG), BSP, Octree, Rasterization Vs Ray tracing

Constructive Solid Geometry (CSG)

Constructive Solid Geometry (CSG)

- A CSG model is *based on the topological notion that a physical object can be divided into a set of primitives (basic elements or shapes) that can be combined in a certain order following a set of rules (Boolean operations) to form the object.*
- Each primitive is bounded by a set of surfaces; usually closed and orientable.
- It is based on the idea of providing a set of predefined object types such as cubes, cone, sphere etc.
- A solid model is created by retrieving primitive solids and performing Boolean operations.

Constructive Solid Geometry (CSG)

Three types of Boolean operations:

- **Union (join):** the operation combines two volumes included in the different solids into a single solid.
- **Subtract (cut):** the operation subtracts the volume of one solid from the other solid object.
- **Intersection:** the operation keeps only the volume common to both solids.

Binary Space Partitioning (BSP)

Binary Space Partitioning (BSP)

- Binary space partitioning is a *3-D graphics programming technique of dividing a scene into two recursively using hyperplanes.*
- In other words, a 3-D scene is split in two using a 2-D plane, then that scene is divided in two using a 2-D plane, and so on. The resulting data structure is a binary tree, or a tree where every node has two branches.
- The technique is *widely used to speed up rendering of 3-D scenes, especially in games.*
- Binary space partitioning was developed in the context of 3D computer graphics, where the structure of a BSP tree allows spatial information about the objects in a scene that is useful in rendering, such as their ordering from front-to-back with respect to a viewer at a given location, to be accessed rapidly.

Binary Space Partitioning (BSP)

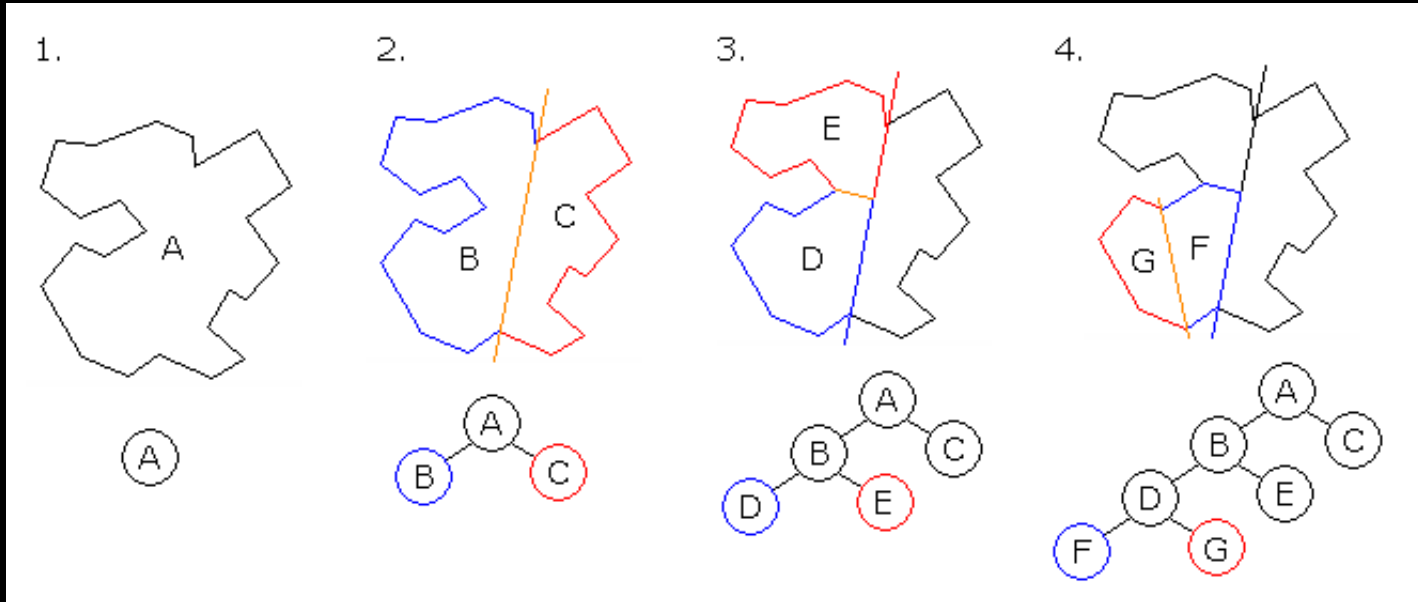
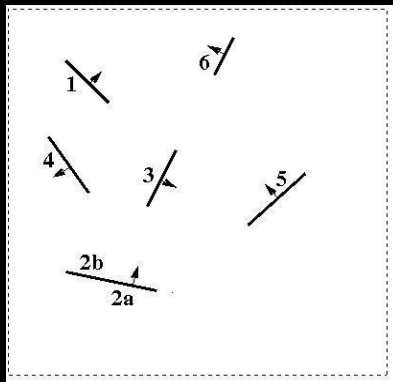


Fig. Binary Space Partitioning (BSP)

Building a BSP Tree (Recursive)

1. Start with a set of polygons and an empty tree
2. Select one of them and make it the root of the tree
3. Use its **plane to divide** the rest of the polygons in 3 sets: front, back, coplanar.
 1. Any polygon crossing the plane is split
4. Repeat the process recursively with the front and back sets, creating the front and back subtrees respectively

Building a BSP Tree (Recursive)



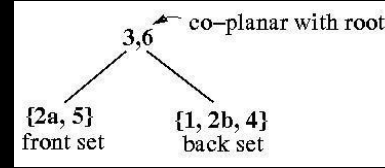
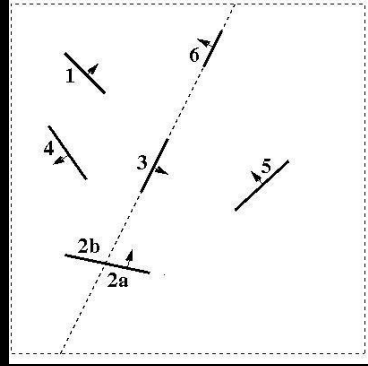
A set of polygons

The tree

{1, 2, 3, 4, 5, 6}

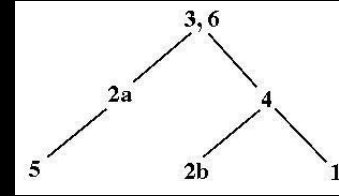
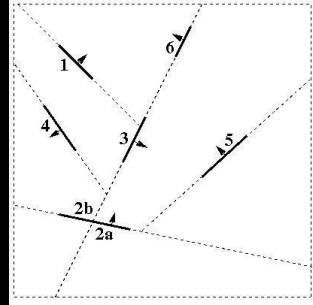
NOTE: If there are N set of polygons then there are N-1 planes

Building a BSP Tree (Recursive)



Select one polygon and partition the space and the polygons

Building a BSP Tree (Recursive)



Recursively partition each sub-tree until all polygons are used up

Applications of Binary Space Partitioning (BSP)

1. **Rendering:** Efficient visibility determination in rendering algorithms.
2. **Collision Detection:** Quick detection of intersections between objects.
3. **Hidden Surface Removal:** Elimination of non-visible surfaces for rendering optimization.
4. **Spatial Partitioning:** Organization and indexing of spatial data in databases and GIS.
5. **Ray Tracing:** Acceleration of ray-object intersection tests for realistic image generation.
6. **Level Design in Games:** Partitioning game worlds for efficient rendering and occlusion culling.
7. **Constructive Solid Geometry (CSG):** Representation of complex geometries using Boolean operations.
8. **Robotics and Path Planning:** Partitioning of workspaces for collision-free path planning in robotics.

Pros of Binary Space Partitioning (BSP)

1. **Efficient Visibility Determination:** BSP trees facilitate fast determination of visible surfaces in rendering algorithms.
2. **Accurate Collision Detection:** BSP trees enable precise detection of intersections between objects, crucial for collision detection algorithms.
3. **Optimized Rendering:** BSP trees support hidden surface removal, improving rendering performance by eliminating non-visible geometry.
4. **Hierarchical Spatial Partitioning:** BSP trees provide a hierarchical structure for organizing spatial data efficiently.
5. **Adaptable to Dynamic Scenes:** BSP trees can adapt to changes in the scene dynamically, making them suitable for dynamic environments.
6. **Useful in Ray Tracing:** BSP trees accelerate ray-object intersection tests, enhancing the efficiency of ray tracing algorithms.
7. **Facilitates Level Design:** BSP trees aid in level design for games by partitioning game worlds into convex regions, optimizing rendering.

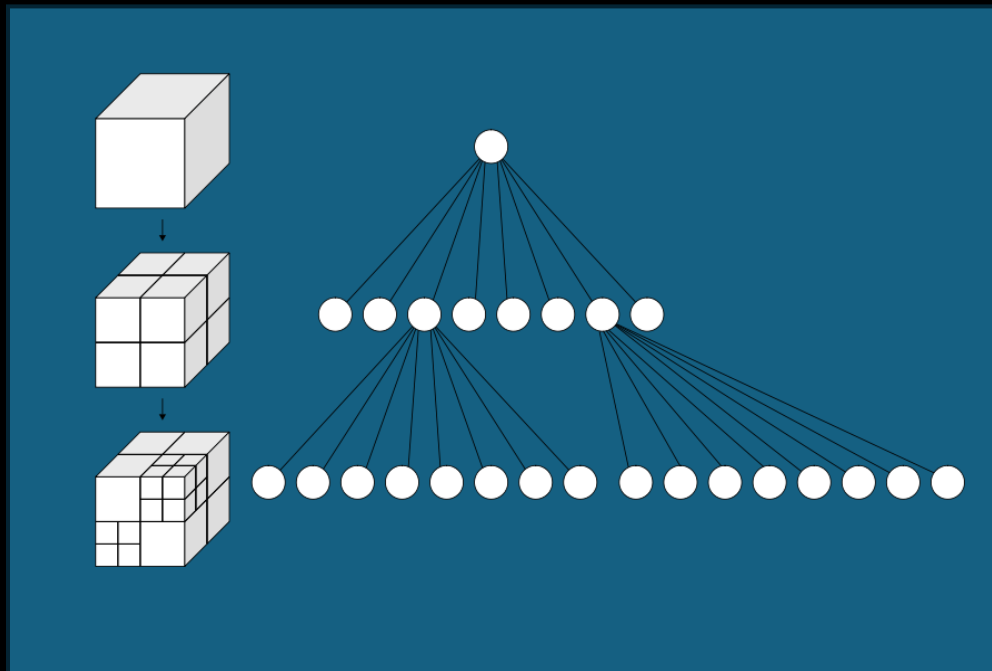
Cons of Binary Space Partitioning (BSP)

1. **Complexity:** Implementing and managing BSP trees can be complex, especially for dynamic scenes or irregular geometries.
2. **Memory Overhead:** BSP trees may consume significant memory, particularly for large or densely populated scenes.
3. **Construction Overhead:** Building BSP trees from a set of objects can involve high computational overhead, impacting performance.
4. **Difficulty in Dynamic Updates:** Modifying BSP trees dynamically can be challenging and may require costly reorganization.
5. **Potential for Imbalanced Trees:** BSP trees may become imbalanced, affecting query performance and memory usage.
6. **Limited in Non-Convex Scenes:** BSP trees are less effective in handling non-convex scenes, potentially leading to suboptimal visibility determination.
7. **Traversal Overhead:** Traversing BSP trees can incur overhead, especially in scenarios with deep trees or complex geometries.

Spatial Partitioning Representation

Spatial Partitioning Representation

- A solid is represented as a collection of adjoining non-intersecting object. For example: Octree



Octree

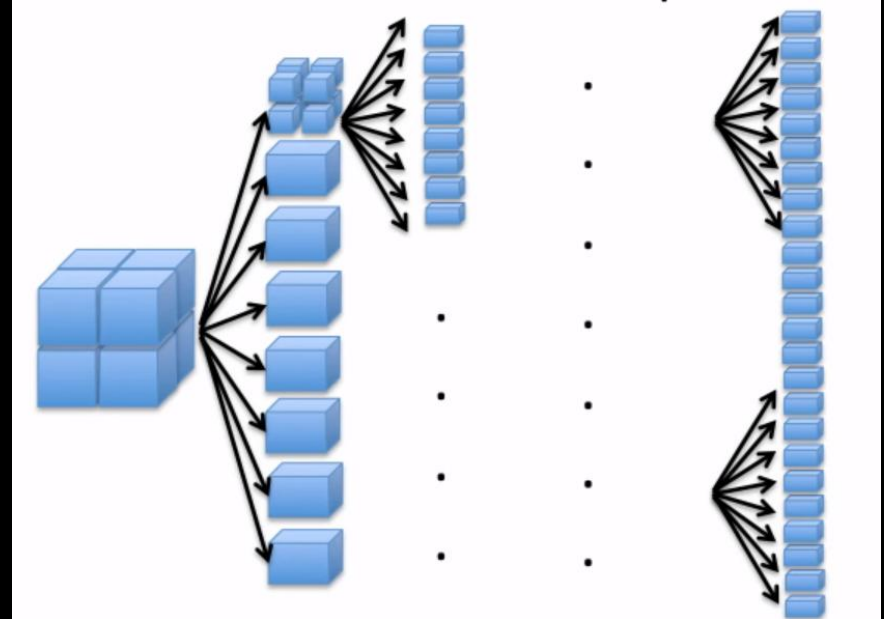
Octree

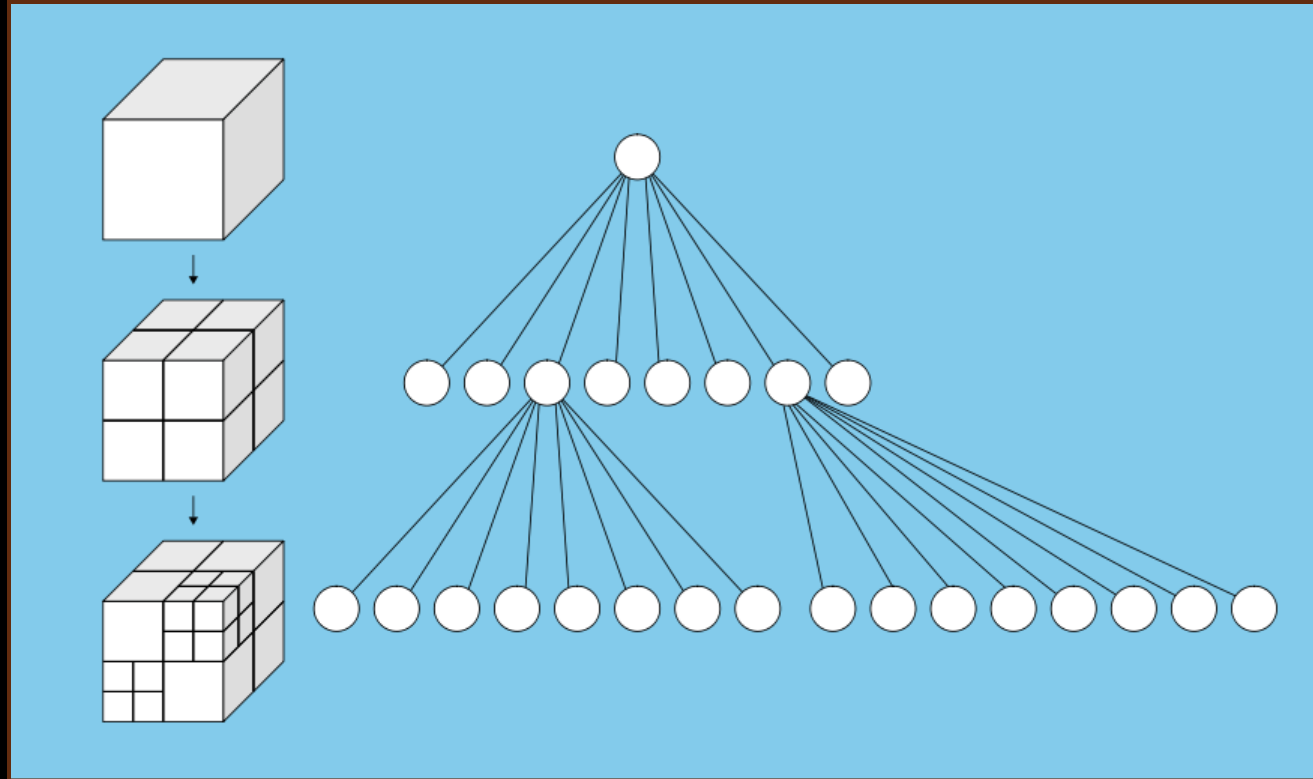
- **Octree** *is a tree data structure in which each internal node has exactly eight children.*
Octrees are most *often used to partition a three-dimensional space by recursively subdividing it into eight octants.*
- Octrees are the three-dimensional analog of quadtrees. The name is formed from *oct* + *tree*, but note that it is normally written "*octree*" with only one "t".
- Octrees are often used in 3D graphics and 3D game engines.

Octree

The Octree can be formed from 3D volume by doing the following steps:

1. Divide the current 3D volume into eight boxes.
2. If any box has more than one point, then divide it further into boxes.
3. Do not divide the box which has one or zero points in it.
4. Do this process repeatedly until all the box contains one or zero point in it.





Octree

Pros:

1. Efficient spatial partitioning.
2. Flexibility in adapting to dynamic scenes.
3. Memory efficiency, especially with sparse data.
4. Effective management of level of detail.
5. Hierarchical representation for fast spatial indexing.

Cons:

1. Memory overhead, especially for large scenes.
2. Complexity in implementation and management.
3. Performance trade-offs, especially in dynamic scenes.
4. Potential resolution issues in dense regions.
5. Construction overhead during tree building.

Rasterization Vs Ray tracing

- **Rasterization** is a rendering method that takes 3D geometric models (made of triangles) and projects them directly onto a 2D pixel grid on your screen.
- **Ray tracing** is a rendering method that simulates the physical behavior of light by tracing individual paths of light rays backward from the virtual camera (the viewer's eye) into the 3D scene.

Basis	Rasterization	Ray tracing
Main Concept	Projects 3D geometry onto a 2D screen grid.	Traces physical paths of light rays through pixels.
Realism	Moderate	High - mimics real-world physics accurately.
Performance	Extremely fast with low computational cost.	Very slow and computationally demanding.
Uses	Real-time video games and interactive apps.	CGI animated movies and high-end gaming visuals.

Questions:

1. Define solid modeling. Why is solid modeling needed in computer graphics ? List down its pros.
2. **Write short notes on:**
 - i. Boundary representation
 - ii. Constructive Solid Geometry (CSG)
 - iii. Sweep representation
 - iv. BSP
 - v. Octree

THANK YOU
Any Queries ?